

ABC457 E - Crossing Table Cloth 题解

题意概括

有 M 块布，第 i 块布恰好覆盖区间 $[L_i, R_i]$ 。

每次询问给出区间 $[S, T]$ ，要求判断能否恰好选出两块布，使得：

- $[S, T]$ 内的每个格子都至少被覆盖一次；
- $[S, T]$ 之外没有任何格子被覆盖。

也就是说，两块布的并集必须恰好等于 $[S, T]$ 。

解题思路

1. 先分成两种情况

对于一个询问 $[S, T]$ ，分两种情况讨论：

1. 存在一块布恰好就是 $[S, T]$
2. 不存在任何一块布恰好是 $[S, T]$

这两种情况的判定方式不同。

2. 情况一：存在一块布恰好覆盖 $[S, T]$

如果已经有一块布正好是 $[S, T]$ ，那么第二块布必须完全包含在 $[S, T]$ 内，否则并集会跑到区间外面。

此时答案为 Yes 的情况一共有三种：

- 恰好为 $[S, T]$ 的布有至少两块；
- 存在一块布完全落在 $[S + 1, T]$ 内；
- 存在一块布完全落在 $[S, T - 1]$ 内。

为什么只要检查这三种？

因为如果第二块布也包含在 $[S, T]$ 内，但又不等于 $[S, T]$ ，那么它一定至少在左端缩进去一格，或者在右端缩进去一格。也就是它一定属于后两类之一。

为了快速判断“是否存在一块左端至少为某个值、右端不超过某个值的布”，可以预处理：

- $\text{min_r_at_l}[i]$ ：左端恰好为 i 的所有布中，最小的右端；
- $\text{suf_min_r}[i]$ ：左端至少为 i 的所有布中，最小的右端。

这样：

- 存在布落在 $[S + 1, T]$ 内，等价于 $\text{suf_min_r}[S + 1] \leq T$
- 存在布落在 $[S, T - 1]$ 内，等价于 $\text{suf_min_r}[S] \leq T - 1$

3. 情况二：不存在任何一块布恰好覆盖 $[S, T]$

这时两块布都不能单独等于 $[S, T]$ ，但它们的并集必须刚好补成整个区间。

由于最终并集左端必须是 S ，所以至少有一块布的左端必须等于 S 。

同理，至少有一块布的右端必须等于 T 。

于是第一块布一定从所有左端为 S 的布里选，第二块布一定从所有右端为 T 的布里选。

进一步地，可以证明：

- 第一块布只需要考虑“左端为 S ，且右端不超过 T 的布里，右端最大的那块”
- 第二块布只需要考虑“右端为 T ，且左端不小于 S 的布里，左端最小的那块”

因为：

- 第一块布右端越靠右越好，越容易和第二块拼起来；
- 第二块布左端越靠左越好，越容易和第一块拼起来。

设这两块布分别是：

- 第一块： $[S, r_1]$
- 第二块： $[l_2, T]$

那么它们能否拼成完整区间，只需要判断：

- 是否存在这样的两块布；
- 并且是否满足 $l_2 \leq r_1 + 1$

因为只要两段相交或相邻，它们的并集就是完整的 $[S, T]$ ；否则中间会出现空缺。

4. 需要维护哪些数据

为了支持上面的判定，可以预处理：

- $\text{cloth_by_l}[i]$ ：把所有布按 (L_i, R_i) 排序后的结果；
- $\text{cloth_by_r}[i]$ ：把所有布按 (R_i, L_i) 排序后的结果；
- $\text{first_by_l}[L]$ 、 $\text{last_by_l}[L]$ ：左端恰好为 L 的布，在 cloth_by_l 中对应的连续区间；
- $\text{first_by_r}[R]$ 、 $\text{last_by_r}[R]$ ：右端恰好为 R 的布，在 cloth_by_r 中对应的连续区间；
- $\text{min_r_at_l}[i]$ ：左端恰好为 i 的布中，最小右端；
- $\text{suf_min_r}[i]$ ：左端至少为 i 的布中，最小右端。

每次询问时：

- 先在 `cloth_by_l` 里，找到左端为 S 的那一段，再二分统计右端恰好等于 T 的布有多少块；
- 若这个数量大于 0，按情况一判定；
- 否则，继续在左端为 S 的那一段里二分出最大的可用右端，在右端为 T 的那一段里二分出最小的可用左端，再检查能否拼起来。

由于所有端点都落在 $1 \sim N$ 内，本题最终不需要 `vector`、`map` 这类容器。

直接把所有布分别排成两个全局数组，再用“某个左端 / 右端在排序数组中的起止位置”来做二分，既满足数组写法，也和题解中的判定过程完全对应。

正确性说明

情况一的正确性

若存在一块布恰好为 $[S, T]$ ，要想恰好选两块并且并集仍然是 $[S, T]$ ，那么第二块布必须完全包含在 $[S, T]$ 内。

如果第二块布也等于 $[S, T]$ ，那么需要至少两块完全相同的布。

如果第二块布不等于 $[S, T]$ ，那么它至少会在左端缩进去，或者在右端缩进去，因此一定满足：

- 完全落在 $[S + 1, T]$ 内，或者
- 完全落在 $[S, T - 1]$ 内。

所以情况一中列出的三种条件恰好覆盖了全部可能，判定正确。

情况二的正确性

若不存在任何一块布恰好为 $[S, T]$ ，而两块布的并集又必须恰好是 $[S, T]$ ，那么：

- 并集的左端必须由某块布提供，因此一定有一块布左端为 S ；
- 并集的右端必须由某块布提供，因此一定有一块布右端为 T 。

设所有左端为 S 且右端不超过 T 的候选里，右端最大的为 r_1 ；

设所有右端为 T 且左端不小于 S 的候选里，左端最小的为 l_2 。

如果这两个最优候选都不能满足 $l_2 \leq r_1 + 1$ ，那么任何别的选择只会让第一块更短，或者第二块更靠右，因此也不可能拼成完整区间。

反过来，只要满足 $l_2 \leq r_1 + 1$ ，那么这两块布相交或相邻，它们的并集就恰好是 $[S, T]$ 。

因此情况二的贪心选择和判定条件都是正确的。

复杂度

- 时间复杂度: $O(N + M \log M + Q \log M)$
- 空间复杂度: $O(N + M)$

其中:

- 预处理排序 by_l 、 by_r 需要 $O(M \log M)$;
- 每个询问只做常数次二分和数组查询, 所以是 $O(\log M)$ 。

参考实现

```
#include <bits/stdc++.h>
using namespace std;

const int MAXN = 200000 + 5;
const int INF = (int)1e9;

struct ClothByLeft{
    int l, r;
} cloth_by_l[MAXN];

struct ClothByRight{
    int r, l;
} cloth_by_r[MAXN];

// first_by_l[l], last_by_l[l] 表示左端为 l 的布, 在排序数组中的范围
// first_by_r[r], last_by_r[r] 表示右端为 r 的布, 在排序数组中的范围
int first_by_l[MAXN];
int last_by_l[MAXN];
int first_by_r[MAXN];
int last_by_r[MAXN];

// min_r_at_l[l] = 左端恰好为 l 的布中, 最小的右端
// suf_min_r[l] = 左端至少为 l 的布中, 最小的右端
int min_r_at_l[MAXN];
int suf_min_r[MAXN];

bool cmp_left(ClothByLeft a, ClothByLeft b){
    if(a.l != b.l){
        return a.l < b.l;
    }
}
```

```
    return a.r < b.r;
}
```

```
bool cmp_right(ClothByRight a, ClothByRight b){
```

```
    if(a.r != b.r){
        return a.r < b.r;
    }
    return a.l < b.l;
}
```

```
// 在 [left, right] 这一段里, 找第一个右端  $\geq$  target 的位置
```

```
int first_r_ge(int left, int right, int target){
```

```
    int ans = right + 1;
    while(left ≤ right){
        int mid = (left + right) / 2;
        if(cloth_by_l[mid].r ≥ target){
            ans = mid;
            right = mid - 1;
        } else {
            left = mid + 1;
        }
    }
    return ans;
}
```

```
// 在 [left, right] 这一段里, 找第一个右端 > target 的位置
```

```
int first_r_gt(int left, int right, int target){
```

```
    int ans = right + 1;
    while(left ≤ right){
        int mid = (left + right) / 2;
        if(cloth_by_l[mid].r > target){
            ans = mid;
            right = mid - 1;
        } else {
            left = mid + 1;
        }
    }
    return ans;
}
```

```
}
```

```
// 在 [left, right] 这一段里, 找第一个左端  $\geq$  target 的位置
```

```
int first_l_ge(int left, int right, int target){
```

```
    int ans = right + 1;
```

```
    while(left  $\leq$  right){
```

```
        int mid = (left + right) / 2;
```

```
        if(cloth_by_r[mid].l  $\geq$  target){
```

```
            ans = mid;
```

```
            right = mid - 1;
```

```
        } else {
```

```
            left = mid + 1;
```

```
        }
```

```
    }
```

```
    return ans;
```

```
}
```

```
// 统计区间 [l, r] 一共出现了多少次
```

```
int count_exact(int l, int r){
```

```
    if(first_by_l[l] == 0){
```

```
        return 0;
```

```
    }
```

```
    int left = first_r_ge(first_by_l[l], last_by_l[l], r);
```

```
    int right = first_r_gt(first_by_l[l], last_by_l[l], r);
```

```
    return right - left;
```

```
}
```

```
// 找左端为 l 且右端  $\leq$  limit_r 的布里, 右端最大的一个
```

```
int get_rightmost(int l, int limit_r){
```

```
    if(first_by_l[l] == 0){
```

```
        return -1;
```

```
    }
```

```
    int pos = first_r_gt(first_by_l[l], last_by_l[l], limit_r) - 1;
```

```
    if(pos < first_by_l[l]){
```

```
        return -1;
```

```
    }
```

```

        return cloth_by_l[pos].r;
    }

    // 找右端为 r 且左端  $\geq$  limit_l 的布里，左端最小的一个
    int get_leftmost(int r, int limit_l){

        if(first_by_r[r] == 0){
            return INF;
        }

        int pos = first_l_ge(first_by_r[r], last_by_r[r], limit_l);
        if(pos > last_by_r[r]){
            return INF;
        }
        return cloth_by_r[pos].l;
    }

    int main(){

        ios::sync_with_stdio(false);
        cin.tie(nullptr);

        // 读取格子数和布的数量
        int n, m;
        cin >> n >> m;

        for(int i=1; i<=n + 1; i++){
            min_r_at_l[i] = INF;
        }

        for(int i=1; i<=m; i++){
            int l, r;
            cin >> l >> r;

            cloth_by_l[i] = {l, r};
            cloth_by_r[i] = {r, l};
            min_r_at_l[l] = min(min_r_at_l[l], r);
        }

        // 把所有布分别按（左端，右端）和（右端，左端）排序
        sort(cloth_by_l + 1, cloth_by_l + m + 1, cmp_left);
    }

```

```

sort(cloth_by_r + 1, cloth_by_r + m + 1, cmp_right);

// 记录每个左端 / 右端在排序数组中的连续区间
for(int i=1; i≤m; i++){
    int l = cloth_by_l[i].l;
    if(first_by_l[l] == 0){
        first_by_l[l] = i;
    }
    last_by_l[l] = i;

    int r = cloth_by_r[i].r;
    if(first_by_r[r] == 0){
        first_by_r[r] = i;
    }
    last_by_r[r] = i;
}

// suf_min_r[i] = 左端至少为 i 的所有布中, 最小的右端
suf_min_r[n + 1] = INF;
for(int i=n; i≥1; i--){
    suf_min_r[i] = min(suf_min_r[i + 1], min_r_at_l[i]);
}

// 读取询问数量
int q;
cin >> q;

while(q--){
    int s, t;
    cin >> s >> t;

    bool ok = false;
    int same_cnt = count_exact(s, t);

    // 情况一: 存在一块布恰好为 [s, t]
    if(same_cnt > 0){
        if(same_cnt ≥ 2){
            ok = true;
        }
        if(s + 1 ≤ n && suf_min_r[s + 1] ≤ t){
            ok = true;
        }
    }
}

```

```

    }
    if(suf_min_r[s] ≤ t - 1){
        ok = true;
    }

    cout << (ok ? "Yes" : "No") << '\n';
    continue;
}

// 情况二：不存在 [s, t]，需要从左端为 s 和右端为 t 的布中各选一块
int r1 = get_rightmost(s, t);
int l2 = get_leftmost(t, s);

// 两块布存在，且中间没有空隙，才能拼成 [s, t]
if(r1 ≠ -1 && l2 ≠ INF && l2 ≤ r1 + 1){
    ok = true;
}

cout << (ok ? "Yes" : "No") << '\n';
}

return 0;
}

```